

Contexto

Até agora, você testou o Sonora manualmente: executava o menu, digitava algumas entradas e verificava se a saída parecia correta. Isso funciona no início, mas não acompanha o crescimento do projeto. Conforme o sistema evolui, repetir todos os testes à mão a cada alteração se torna trabalhoso, e um erro antigo pode voltar sem ser percebido.

Teste unitário e o checklist antes de lançar o foguete: um conjunto de verificações automáticas que roda em segundos e te avisa na hora se alguma coisa quebrou. Nesta fase você vai transformar aquele testar no olho em testes de verdade, escritos em JUnit 6, que comprovam o comportamento das suas classes, incluindo as exceções que você adicionou na Fase 02.

E tem um detalhe honesto: se algum teste seu falhar, pode ser que ele tenha achado um bug que estava escondido na sua Fase 02. Ótimo. Era exatamente para isso que ele serve.

Objetivo

1. Clonar o projeto sonora-fase02 para uma pasta nova sonora-fase03.
2. Documentar planos de teste no formato de tabela (modelo mais abaixo).
3. Implementar esses planos como testes automatizados em JUnit 6.
4. Deixar todos os testes passando (verde).

Você não vai reescrever as classes do Sonora. O trabalho aqui é por cima do que já existe: adicionar as classes de teste e, se algum teste revelar um bug, corrigir a classe de produção.

Contrato da Fase 02 (confira antes de começar)

Os planos de teste desta fase assumem o seguinte comportamento, que é o que você implementou na Fase 02. Se o seu projeto ficou diferente em algum ponto, ajuste a saída esperada do plano correspondente.

Situação	Comportamento esperado
Construtor/setter de Música com título vazio, artista vazio ou duração menor ou igual a zero	Lança <code>IllegalArgumentException</code>
Construtor/setter de Usuário com nome vazio ou Email vazio	Lança <code>IllegalArgumentException</code>
<code>Playlist.getNaPosicao(indice)</code> ou <code>removerNaPosicao(indice)</code> com índice fora da faixa	Lança <code>IndexOutOfBoundsException</code>
<code>Playlist.adicionar(música)</code> com a playlist cheia	Retorna false (fluxo normal, não é exceção)
<code>Plataforma.cadastrarMusica / cadastrarUsuario</code> com estrutura cheia	Retorna false (fluxo normal)
<code>Plataforma.buscarMusica(...)</code> / <code>buscarMusicaPorId(...)</code> sem encontrar	Retorna null (fluxo normal)



A ideia por trás da divisão: exceção e para uso indevido do programador (pedir uma posição que não existe, criar objeto inválido). Situação normal de operação (uma playlist encher, uma busca não achar) continua sinalizada por valor de retorno.

Parte 1 - Planos de teste

Antes de sair codando, você documenta o que vai testar. Cada linha de um plano de teste vira depois um método de teste. O formato é este: um título identificando o plano e uma tabela com as colunas Caso, Descrição, Entrada e Saida esperada.

Planos prontos (referência)

Estes dois já vem preenchidos. Use-os como modelo do nível de detalhe esperado e como base para implementar na Parte 2.

Plano de testes PL01 - Validar Musica.getDuracaoFormatada()

Caso	Descrição	Entrada	Saida esperada
1	Duração com minutos e segundos	Música de 125 segundos	Deve resultar em "02:05"
2	Duração redonda em minutos	Música de 90 segundos	Deve resultar em "01:30"
3	Menos de um minuto, com zero a esquerda	Música de 5 segundos	Deve resultar em "00:05"
4	Dois dígitos nos minutos	Música de 600 segundos	Deve resultar em "10:00"
5	Valor logo abaixo de dez minutos	Música de 599 segundos	Deve resultar em "09:59"

Plano de testes PL02 - Validar construtor de Música com dados invalidos

Caso	Descrição	Entrada	Saida esperada
1	Título vazio deve ser rejeitado	título "", artista "Queen", duracao 355	Deve lançar IllegalArgumentException
2	Título nulo deve ser rejeitado	título null, artista "Queen", duracao 355	Deve lançar IllegalArgumentException
3	Artista vazio deve ser rejeitado	título "Bohemian Rhapsody", artista "", duração 355	Deve lançar IllegalArgumentException
4	Duração zero deve ser rejeitada	título valido, artista valido, duracao 0	Deve lançar IllegalArgumentException
5	Duração negativa deve ser rejeitada	título valido, artista valido, duração -10	Deve lançar IllegalArgumentException



Caso	Descrição	Entrada	Saida esperada
6	Dados validos criam a música	título "Bohemian Rhapsody", artista "Queen", duração 355	Objeto criado, com id maior que zero

Planos que você vai montar

Para cada plano abaixo eu te dou apenas o alvo (a classe, o método e os aspectos a cobrir). Você preenche a tabela inteira: Caso, Descrição, Entrada e Saida esperada. Cubra tanto os casos que dão certo quanto os que devem falhar. Pense em pelo menos 3 casos por plano.

- **PL03 - Playlist.adicionar(música).** Cobrir: adicionar em playlist com espaço (retorna true e a quantidade sobe); adicionar até encher a playlist (o que ultrapassa a capacidade retorna false).
- **PL04 - Playlist.getNaPosicao(indice).** Cobrir: posição valida devolve a música certa; índice negativo e índice além da quantidade.
- **PL05 - Playlist.removeNaPosicao(indice).** Cobrir: remoção de uma posição valida reorganiza sem deixar buraco (a música seguinte assume a posição); índice invalido.
- **PL06 - Plataforma: buscarMusica(título) e buscarMusicaPorId(id).** Cobrir: música cadastrada e encontrada; busca por título/id inexistente.
- **PL07 - Musica.reproduzir().** Cobrir: cada chamada aumenta o contador de reproduções em um.
- **PL08 (bônus) - Contadores de id.** Cobrir: ids de Música saem sequenciais (1, 2, 3...) e são independentes dos ids de Usuário.

Parte 2 - Implementação em JUnit 6

Cada caso dos seus planos vira um método de teste. Regras:

1. Use `@Test` em cada método e `@DisplayName` com a descrição do caso (o texto da coluna Descrição). Assim o relatório de testes fica legível e amarra o código ao plano.
2. Casos normais: use `assertEquals`, `assertTrue`, `assertFalse`, `assertNull` ou `assertNotNull`, conforme o caso.
3. Casos de exceção: use `assertThrows`, verificando o tipo exato da exceção. Não basta estourar, tem que estourar a exceção certa.
4. Use `@BeforeEach` para montar o cenário base que se repete (por exemplo, uma Plataforma já com algumas músicas e um usuário cadastrados), em vez de repetir esse preparo em cada método.